

1. Git क्या है? और बताएं कि हम Git का इस्तेमाल क्यों करते हैं?

उत्तर:

Git का परिचय

Git एक पॉपुलर वर्जन कंट्रोल सिस्टम है। इसे 2005 में Linus Torvalds ने बनाया था, और तब से Junio Hamano इसे मेंटेन कर रहे हैं।

के उपयोग में आना:

- ट्रैकिंग कोड में बदलाव
- यह ट्रैक करना कि किसने बदलाव किए
- कोडिंग सहयोग

Git के साथ काम करना

- किसी फ़ोल्डर पर Git इनिशियलाइज़ करें, जिससे वह एक रिपोजिटरी बन जाए
- Git अब उस फ़ोल्डर में बदलावों का ट्रैक रखने के लिए एक छिपा हुआ फ़ोल्डर बनाता है
- जब कोई फ़ाइल बदली, जोड़ी या हटाई जाती है, तो उसे मॉडिफाइड माना जाता है
- आप उन संशोधित फ़ाइलों का चयन करते हैं जिन्हें आप स्टेज करना चाहते हैं
- स्टेज्ड फ़ाइलें कमिटेड हैं, जो Git को एक परमानेंट स्नैपशॉट स्टोर करने के लिए प्रॉम्प्ट करती हैं
- Git आपको हर कमिट की पूरी हिस्ट्री देखने की सुविधा देता है।
- आप किसी भी पिछले कमिट पर वापस जा सकते हैं।
- Git हर कमिट में हर फ़ाइल की एक अलग कॉपी स्टोर नहीं करता है, लेकिन उसका ट्रैक रखता है
- हर कमिट में बदलाव किए गए!

Git क्यों?

- 70% से ज़्यादा डेवलपर्स Git का इस्तेमाल करते हैं!
- डेवलपर्स दुनिया में कहीं से भी एक साथ काम कर सकते हैं।
- डेवलपर्स प्रोजेक्ट का पूरा इतिहास देख सकते हैं।
- डेवलपर्स किसी प्रोजेक्ट के पुराने वर्शन पर वापस जा सकते हैं।

नोट: अधिकांश Git क्रियाएँ (जैसे स्टेजिंग, कमिटिंग और देखने)

2. Git में वर्जन कंट्रोल समझाएं?

1. वर्जन कंट्रोल का ओवरव्यू

वर्जन कंट्रोल सिस्टम (VCS) एक बेसिक सॉफ्टवेयर टूल है जिसे सोर्स कोड या किसी भी फाइल के सेट में समय के साथ होने वाले बदलावों को मैनेज और ट्रैक करने के लिए डिज़ाइन किया गया है। यह एक बड़े डेटाबेस की तरह काम करता है जो किसी प्रोजेक्ट में किए गए हर बदलाव को रिकॉर्ड करता है, जिसमें यह भी शामिल है कि बदलाव किसने किया, क्या हुआ।

बदला गया है, और क्यों। अगर कोई डेवलपर कोई बड़ी गलती करता है, तो VCS टीम को दूसरे टीम मेंबर्स के चल रहे काम में रुकावट डाले बिना, आसानी से पिछली, स्टेबल हालत में वापस जाने देता है।

मॉडर्न वर्शन कंट्रोल के आने से पहले, डेवलपर्स मैनुअल फ़ाइल मैनेजमेंट पर निर्भर थे, अक्सर `project_final.docx`, `project_final_final_v2.docx`, और `project_USE_THIS_FINAL.docx` जैसे अजीब नाम रखने के तरीकों में। इस मैनुअल तरीके से अक्सर कोड ओवरराइट हो जाता था, इस बात की जवाबदेही नहीं होती थी कि खास बग किसने किए, और कई डेवलपर्स के काम को मर्ज करने में बहुत मुश्किल होती थी। एक मज़बूत VCS सच्चाई का एक ही सोर्स बनाए रखकर और पुराने बदलावों की ट्रैकिंग को ऑटोमेट करके इन दिक्कतों को खत्म करता है।

वर्जन कंट्रोल का विकास तीन मुख्य फेज़ से हुआ है। शुरुआती लोकल VCS में बस फाइलों को एक बैकअप डायरेक्टरी में कॉपी किया जाता था। सेंट्रलाइज्ड VCS (जैसे SVN) ने सभी वर्जन वाली फाइलों को रखने के लिए एक सिंगल सर्वर शुरू किया, लेकिन अगर सर्वर डाउन हो जाता तो इससे फेलियर का एक सिंगल पॉइंट बन जाता था। आज, इंडस्ट्री स्टैंडर्ड डिस्ट्रिब्यूटेड VCS (DVCS) है, जिसका सबसे बड़ा उदाहरण Git है। DVCS में, हर डेवलपर के कंप्यूटर में पूरी रिपॉजिटरी का एक पूरा मिरर होता है, जिसमें उसकी पूरी हिस्ट्री भी शामिल होती है। इसका मतलब है कि अगर सेंट्रल सर्वर फेल हो जाता है, तो किसी भी डेवलपर की लोकल कॉपी का इस्तेमाल उसे रिस्टोर करने के लिए किया जा सकता है, जो बेमिसाल रेजिलिएंस और ऑफलाइन काम करने की क्षमता देता है।

3. Github के बारे में विस्तार से बताएं?

GitHub क्या है?

GitHub एक वेब-बेस्ड होस्टिंग सर्विस और कोलेबोरेशन प्लेटफॉर्म है जो खास तौर पर Git रिपॉजिटरी के लिए बनाया गया है। यह एक कंपनी है (जिसका मालिकाना हक माइक्रोसॉफ्ट के पास है) जो एक वेबसाइट और क्लाउड सर्वर देती है।

- यह क्या करता है: यह आपकी Git रिपॉजिटरी को इंटरनेट पर (क्लाउड में) स्टोर करता है। यह एक रिमोट बैकअप की तरह काम करता है और दुनिया भर के कई डेवलपर्स को एक ही प्रोजेक्ट पर एक साथ काम करने देता है।
- यह कहाँ रहता है: यह इंटरनेट पर रहता है (github.com पर)। आपको अपना कोड इस पर पुश करने या इससे कोड खींचने के लिए इंटरनेट कनेक्शन की ज़रूरत होती है।
- अतिरिक्त सुविधाएँ: GitHub, Git की कच्ची शक्ति लेता है और एक उपयोगकर्ता के अनुकूल इंटरफ़ेस और शक्तिशाली सहयोग उपकरण जोड़ता है, जैसे:
 - o पुल रिक्वेस्ट (PRs): कोड में बदलाव का प्रस्ताव देने और उसे रिव्यू करने का एक तरीका उन्हें मिलाना।
 - o समस्याएँ: एक बिल्ट-इन टास्क और बग-ट्रैकिंग सिस्टम।
 - o GitHub Actions: टेस्टिंग और डिप्लॉयमेंट (CI/CD) को ऑटोमेट करने के लिए टूल्स।
 - o पोर्टफोलियो: यह डेवलपर्स के लिए अपना कोड दिखाने के लिए एक पब्लिक रिज्यूमे का काम करता है नियोक्ताओं।

4. विंडोज ऑपरेटिंग सिस्टम में git इंस्टॉल करने के स्टेप्स बताएं?

2. Git इंस्टॉल करना और GitHub अकाउंट सेटअप करना

स्ट्रैटेजी के लिए Git और GitHub के बीच का अंतर समझना बहुत ज़रूरी है। Git असल में वर्जन कंट्रोल सॉफ्टवेयर है जो आपकी मशीन पर लोकल तौर पर चलता है, और आपकी फाइलों और हिस्ट्री को ट्रैक करता है।

दूसरी तरफ, GitHub एक क्लाउड-बेस्ड होस्टिंग सर्विस है जो इंटरनेट पर Git रिपॉजिटरी स्टोर करती है, जिससे कोलेबोरेशन और बैकअप हो पाता है। शुरू करने के लिए, आपको Git सॉफ्टवेयर इंस्टॉल करना होगा।

Windows यूज़र्स को ऑफिशियल git-scm.com वेबसाइट से इंस्टॉलर डाउनलोड करना चाहिए और इसे डिफ़ॉल्ट सेटिंग्स के साथ चलाना चाहिए। macOS यूज़र्स इसे Homebrew (brew install git) का इस्तेमाल करके टर्मिनल से या ऑफिशियल इंस्टॉलर डाउनलोड करके इंस्टॉल कर सकते हैं, जबकि Linux यूज़र्स अपने डिस्ट्रीब्यूशन के पैकेज मैनेजर, जैसे Ubuntu के लिए sudo apt install git का इस्तेमाल कर सकते हैं। इंस्टॉल होने के बाद, आप अपना टर्मिनल या कमांड प्रॉम्प्ट खोलकर और git --version टाइप करके इंस्टॉलेशन को वेरिफाई कर सकते हैं, जिससे एक वर्जन नंबर मिलना चाहिए।

Git इंस्टॉल होने के बाद, अगला स्टेप GitHub अकाउंट बनाना है। github.com पर जाएं और फ्री अकाउंट के लिए रजिस्टर करें। यूज़रनेम चुनते समय, एक प्रोफेशनल आइडेंटिफायर, जैसे आपका असली नाम या उसका कोई दूसरा रूप (जैसे, rahul-sharma या dev-rahul) इस्तेमाल करने की सलाह दी जाती है। इनफॉर्मल निकनेम इस्तेमाल करने से बचें, क्योंकि आपकी GitHub प्रोफाइल आपकी कोडिंग जर्नी का एक पब्लिक पोर्टफोलियो होगी और कैपस प्लेसमेंट के दौरान पोर्टेबिल एम्प्लॉयर इसे रिव्यू करेंगे।

फ्री टियर एकेडमिक प्रोजेक्ट्स, पर्सनल पोर्टफोलियो और ओपन-सोर्स कंट्रीब्यूशन के लिए सभी ज़रूरी फीचर्स देता है।

5. Git के अलग-अलग एप्लीकेशन के बारे में बताएं?

उत्तर:

Git के अनुप्रयोग और उपयोग

Git को मुख्य रूप से सॉफ्टवेयर डेवलपमेंट के लिए एक वर्जन कंट्रोल सिस्टम के तौर पर जाना जाता है, लेकिन इसका इस्तेमाल सिर्फ कोड लिखने से कहीं ज़्यादा है। Git को अलग-अलग डोमेन में कैसे इस्तेमाल किया जाता है, यहाँ बताया गया है:

1. सॉफ्टवेयर डेवलपमेंट और सोर्स कोड मैनेजमेंट

- कोलेबोरेटिव कोडिंग: यह कई डेवलपर्स को एक ही कोडबेस पर एक-दूसरे के काम को ओवरराइट किए बिना एक साथ काम करने देता है।
- कोड रिव्यू: नए कोड से पहले पुल रिक्वेस्ट (PRs) के ज़रिए पीयर रिव्यू की सुविधा देता है
मेन प्रोजेक्ट में मर्ज कर दिया गया है।
- रिलीज़ प्रबंधन: टीमों को सॉफ्टवेयर के विभिन्न संस्करणों (जैसे, v1.0, v1.1, बीटा रिलीज़) टैग और ब्रांच का इस्तेमाल करके।

2. DevOps और कंटीन्यूअस इंटीग्रेशन/कंटीन्यूअस डिप्लॉयमेंट (CI/CD)

- ऑटोमेटेड पाइपलाइन: Git रिपॉजिटरी CI/CD टूल्स (जैसे GitHub Actions, Jenkins, या GitLab CI) के लिए ट्रिगर का काम करती हैं। हर बार जब कोड पुश किया जाता है, तो ऑटोमेटेड टेस्ट और डिप्लॉयमेंट ट्रिगर हो सकते हैं।
- इंफ्रास्ट्रक्चर ऐज़ कोड (IaC): इसका इस्तेमाल इंफ्रास्ट्रक्चर कॉन्फिगरेशन फ़ाइलों (जैसे, Dockerfiles, Terraform स्क्रिप्ट, Ansible प्लेबुक) में बदलावों को ट्रैक करने के लिए किया जाता है, यह पक्का करता है कि इंफ्रास्ट्रक्चर में बदलाव सॉफ्टवेयर की तरह ही वर्शन में हों।

3. टेक्निकल डॉक्यूमेंटेशन और कंटेंट मैनेजमेंट

- डॉक्यूमेंटेशन ट्रैकिंग: टेक्निकल राइटर मैनुअल, API डॉक्यूमेंटेशन और README फाइलों में बदलावों को ट्रैक करने के लिए Git का इस्तेमाल करते हैं।
- किताब और आर्टिकल लिखना: LaTeX में लिखी कई टेक्निकल किताबें (मशहूर "Pro Git" किताब समेत) और एकेडमिक पेपर, एडिट को ट्रैक करने और को-ऑथर्स के साथ मिलकर काम करने के लिए Git का इस्तेमाल करके वर्जन-कंट्रोल होते हैं।

4. कॉन्फिगरेशन प्रबंधन

- सिस्टम एडमिनिस्ट्रेशन: सिस्टम एडमिनिस्ट्रेटर सर्वर कॉन्फिगरेशन फ़ाइलों (जैसे .bashrc, Nginx कॉन्फिगरेशन, या फ़ायरवॉल नियम) में किए गए बदलावों को ट्रैक करने के लिए Git का इस्तेमाल करते हैं, जिससे वे किसी खराब कॉन्फिगरेशन को आसानी से रोल बैक कर सकते हैं जिससे सर्वर क्रैश हो सकता है।

5. एकेडमिक रिसर्च और डेटा साइंस

- रिसर्च रिप्रोड्यूसिबिलिटी: रिसर्चर अपनी डेटा एनालिसिस स्क्रिप्ट को वर्जन-कंट्रोल करने के लिए Git का इस्तेमाल करते हैं, जिससे यह पक्का होता है कि रिसर्च पेपर के रिजल्ट बनाने के लिए इस्तेमाल किया गया एकदम सही कोड सुरक्षित रहे।
- डेटासेट वर्शनिंग: हालांकि Git बड़ी बाइनरी फ़ाइलों के लिए नहीं है, लेकिन Git पर बने टूल (जैसे Git LFS - लार्ज फ़ाइल स्टोरेज) डेटा साइटिस्ट को डेटासेट और मशीन लर्निंग मॉडल में होने वाले बदलावों को ट्रैक करने में मदद करते हैं।

6. Git के फीचर्स क्या हैं?

उत्तर:

Git की मुख्य विशेषताएं

इंडस्ट्री में Git का दबदबा इसके दमदार, खास फीचर्स की वजह से है। Git की खासियतें ये हैं:

1. वितरित आर्किटेक्चर

- पूरी लोकल कॉपी: सेंट्रलाइज़्ड सिस्टम के उलट, हर डेवलपर पूरी कॉपी बनाता है रिपॉजिटरी, जिसमें इसकी पूरी हिस्ट्री शामिल है।
- ऑफ़लाइन कैपेबिलिटी: क्योंकि पूरी हिस्ट्री लोकल होती है, इसलिए डेवलपर्स बिना इंटरनेट कनेक्शन के कमिट कर सकते हैं, हिस्ट्री देख सकते हैं, ब्रांच बना सकते हैं और मर्ज कर सकते हैं। उन्हें सेंट्रल सर्वर पर बदलाव पुश/पुल करने के लिए सिर्फ़ नेटवर्क की ज़रूरत होती है।
- कोई सिंगल पॉइंट ऑफ़ फेलियर नहीं: अगर सेंट्रल GitHub/GitLab सर्वर डाउन हो जाता है, तो किसी भी डेवलपर की लोकल कॉपी का इस्तेमाल सेंट्रल सर्वर को रिस्टोर करने के लिए किया जा सकता है।

2. ब्रांचिंग और मर्जिंग

- लाइटवेट ब्रांच: Git में, एक ब्रांच बस एक खास कमिट का पॉइंटर होता है। ब्रांच बनाना, स्विच करना और डिलीट करना लगभग तुरंत होता है और इसमें बहुत कम जगह लगती है। याद।

- सुरक्षित एक्सपेरिमेंटेशन: डेवलपर्स किसी नए फीचर को टेस्ट करने के लिए "सैंडबॉक्स" ब्रांच बना सकते हैं।

अगर यह फेल हो जाता है, तो वे मेन कोडबेस पर असर डाले बिना इसे डिलीट कर सकते हैं। अगर यह सफल होता है, तो इसे आसानी से वापस मर्ज किया जा सकता है।

3. स्टेजिंग एरिया (इंडेक्स)

- सेलेक्टिव कमिटिंग: Git वर्किंग डायरेक्टरी और रिपॉजिटरी के बीच एक बीच का एरिया लाता है। इससे डेवलपर्स किसी फ़ाइल या खास फ़ाइलों के सिर्फ़ खास हिस्सों को स्टेज कर सकते हैं, जिससे बहुत ज़्यादा फोकस्ड और लॉजिकल कमिट बनते हैं, भले ही वे एक ही समय में कई अलग-अलग चीज़ों पर काम कर रहे हों।

4. डेटा इंटीग्रिटी और सिक्योरिटी

- क्रिप्टोग्राफ़िक हैशिंग: Git में सब कुछ (फ़ाइलें, डायरेक्टरी, कमिट) स्टोर करने से पहले क्रिप्टोग्राफ़िक हैश (SHA-1, और SHA-256 में ट्रांज़िशन) का इस्तेमाल करके चेकसम किया जाता है।

- टैम्पर-एविडेंट: Git को पता चले बिना किसी भी फ़ाइल या कमिट का कंटेंट बदलना फिजिकली नामुमकिन है। अगर कोई फ़ाइल ट्रांज़िट में करप्ट हो जाती है या उसके साथ छेड़छाड़ की जाती है, तो Git तुरंत मिसमैच हैश का पता लगाकर उसे रिजेक्ट कर देगा।

5. गैर-रैखिक विकास

- कॉम्प्लेक्स हिस्ट्री: Git को तेज़ी से ब्रांचिंग और मर्जिंग को हैंडल करने के लिए डिज़ाइन किया गया है। यह कॉम्प्लेक्स, नॉन-लीनियर डेवलपमेंट हिस्ट्री को ट्रैक कर सकता है, और हज़ारों अलग-अलग ब्रांच पर एक साथ काम कर रहे सैकड़ों डेवलपर्स के काम को मर्ज कर सकता है।

6. स्पीड और परफॉर्मेंस

- लोकल ऑपरेशन: क्योंकि लगभग सभी ऑपरेशन (कमिटिंग, डिफ़िंग, लॉगिंग, ब्रांचिंग) नेटवर्क के बजाय लोकल डिस्क पर किए जाते हैं, इसलिए Git बहुत तेज़ है।

- ऑप्टिमाइज़्ड बैकएंड: Git का अंदरूनी फ़ाइल सिस्टम और ऑब्जेक्ट मॉडल बड़ी मात्रा में डेटा को तेज़ी से पढ़ने और लिखने के लिए बहुत ज़्यादा ऑप्टिमाइज़्ड हैं।

7. वर्कफ़्लो में फ़्लेक्सिबिलिटी

- टीम के हिसाब से ढलने वाला: Git किसी एक वर्कफ़्लो को फ़ोर्स नहीं करता है। टीम अपनी ज़रूरतों के हिसाब से अलग-अलग स्ट्रेटेजी अपना सकती हैं, जैसे GitFlow (स्ट्रिक्ट रिलीज़ साइकल), फ़ीचर ब्रांच वर्कफ़्लो (हर फ़ीचर को एक ब्रांच मिलती है), या फ़ॉर्किंग वर्कफ़्लो (ओपन-सोर्स में आम है, जहाँ कंट्रीब्यूटर मेन रेपो को फ़ॉर्क करते हैं और PRs सबमिट करते हैं)।

8. ओपन सोर्स और एक्सटेंसिबल

- इस्तेमाल करने के लिए मुफ़्त: Git एक मुफ़्त और ओपन-सोर्स सॉफ़्टवेयर (FOSS) है, जिसका मतलब है कि कोई भी इसे बिना लाइसेंस फ़ीस के किसी भी काम के लिए इस्तेमाल कर सकता है।
- रिच इकोसिस्टम: क्योंकि यह ओपन-सोर्स है, इसलिए इसके आस-पास थर्ड-पार्टी टूल्स, GUI क्लाइंट्स (जैसे सोर्सट्री, गिटक्रैकन), और इंटीग्रेशन्स (जिरा, स्लैक, VS कोड) का एक बहुत बड़ा इकोसिस्टम बना हुआ है।